

CS4231
Parallel and Distributed Algorithms

Lecture 8
Instructor: Haifeng YU

Review of Last Lecture

- Leader election on a ring
 - Chang-Roberts Algorithm
 - Interesting avg performance analysis
- Leader election on a general graph
 - Spanning tree construction

Today's Roadmap

- “Agreement”
 - Also called consensus
- Motivation
- 5 Different Versions of Distributed Consensus

Motivation

- Distributed consensus
 - A set of nodes want to agree on something
 - Each node has an input
 - E.g., whether the flight tickets for segments of your journey should be purchased
- Version 0: No failures
 - Everyone send its input to everyone else
 - Take the majority of the n values (with tie-breaking favoring 0). Decide, and Done.

Motivation

- But things fail
 - Nodes can crash
 - Internet can be down
- Goal: Distributed consensus despite failures
- When there are no failures:
 - Everyone send its input to everyone else
 - Take the majority of the n values (with tie-breaking favoring 0). Decide, and Done.
- What will go wrong if some nodes crash?

Quick Poll <https://pollev.com/haifengyu229>

5 Versions of Distributed Consensus

<u>Failure Model and Timing Model</u>	
Ver0: No node or link failures	Trivial
Ver1: Node crash failures; Channels are reliable; Synchronous;	This lecture
Ver2: No node failures; Channels may drop messages (the coordinated attack problem)	This lecture
Ver3: Node crash failures; Channels are reliable; Asynchronous;	Next lecture
Ver4: Node Byzantine failures; Channels are reliable; Synchronous; (the Byzantine Generals problem)	Next lecture

Distributed Consensus Version 1: Consensus with Node Crash Failures/Synchronous

- Failure model:
 - Nodes may experience **crash failure**
 - Communication channels are reliable
- Timing model
 - **Synchronous**: Message delay has a **known** upper bound x , and node processing delay has a **known** upper bound y
 - Synchronous timing model implies the possibility of accurate failure detection
- Goal:
 - **Termination**: All nodes (that have not failed) eventually decide
 - **Agreement**: All nodes that decide should decide on the same value
 - **Validity**: If all nodes have the same initial input, that value should be the only possible decision value. Otherwise nodes are allowed to decide on anything (except that they still need to satisfy the Agreement requirement)

Synchronous System and Rounds

- Under a synchronous timing model, we can always make all processes proceed in **inter-locked rounds**, where in each round:
 - Every process does some local computation
 - Every process sends one message to every other process
 - Every process receives one message from every other process
 - Why is this general enough?
- Why is round useful? (In fact, VERY useful)
 - Conceptually very clear – All processes make progress round by round, no one is left behind
 - Messages sent during a round is received during that round
 - Accurate failure detection

Implementing Rounds

- Assume that each process has a physical clock with some bounded clock error: E.g., the clock reading on two processes, at any given point of time, differ by at most 1 second
- Assume message propagation delay is at most 3 seconds
- Assume local processing delay at most 4 seconds
- Set round duration to be $1+3+4 = 8$ seconds.
- Each process starts a new round every 8 seconds, according to its local clock.
- A message sent in a round must be received **by** the end of that round on the receiver (and may be much earlier than that)
 - Each message has a round number attached on it.

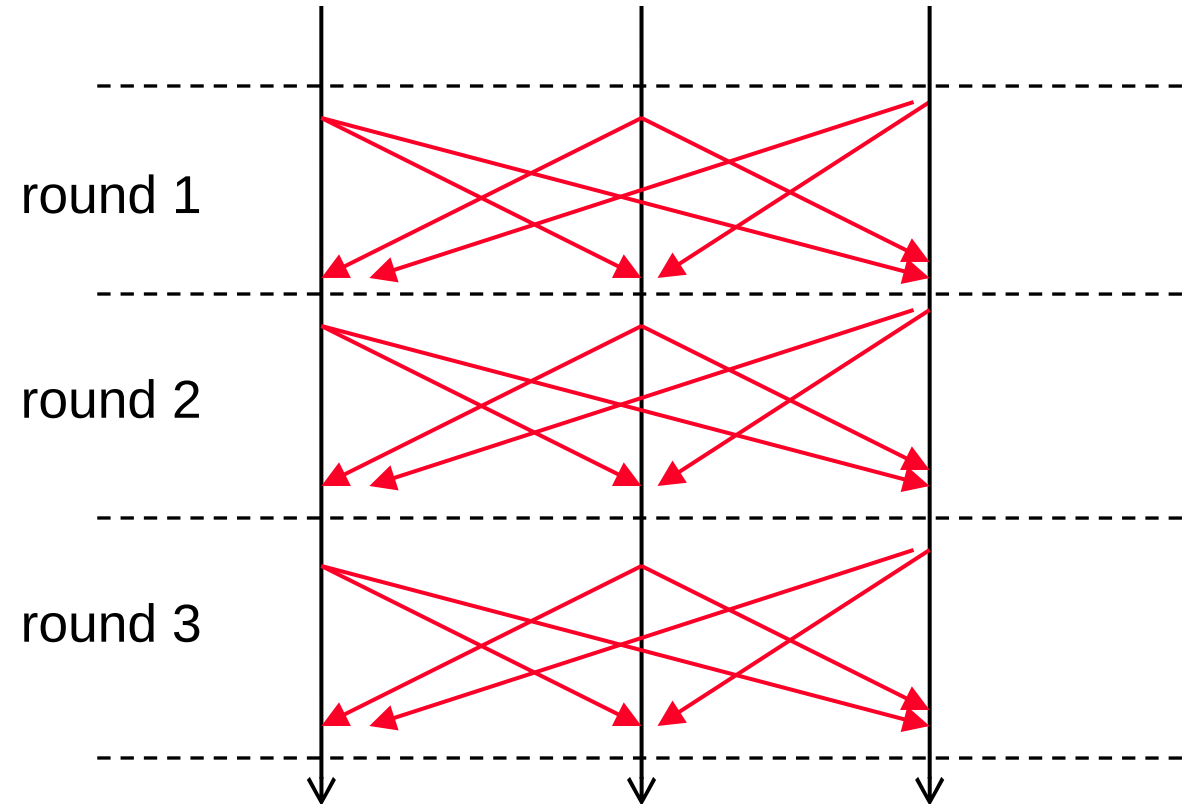
Synchronous System and Rounds

- Only a synchronous system enables us to define a round as above
 - The notion of rounds is very convenient and powerful!
 - But remember that we do need the previous protocol in practice

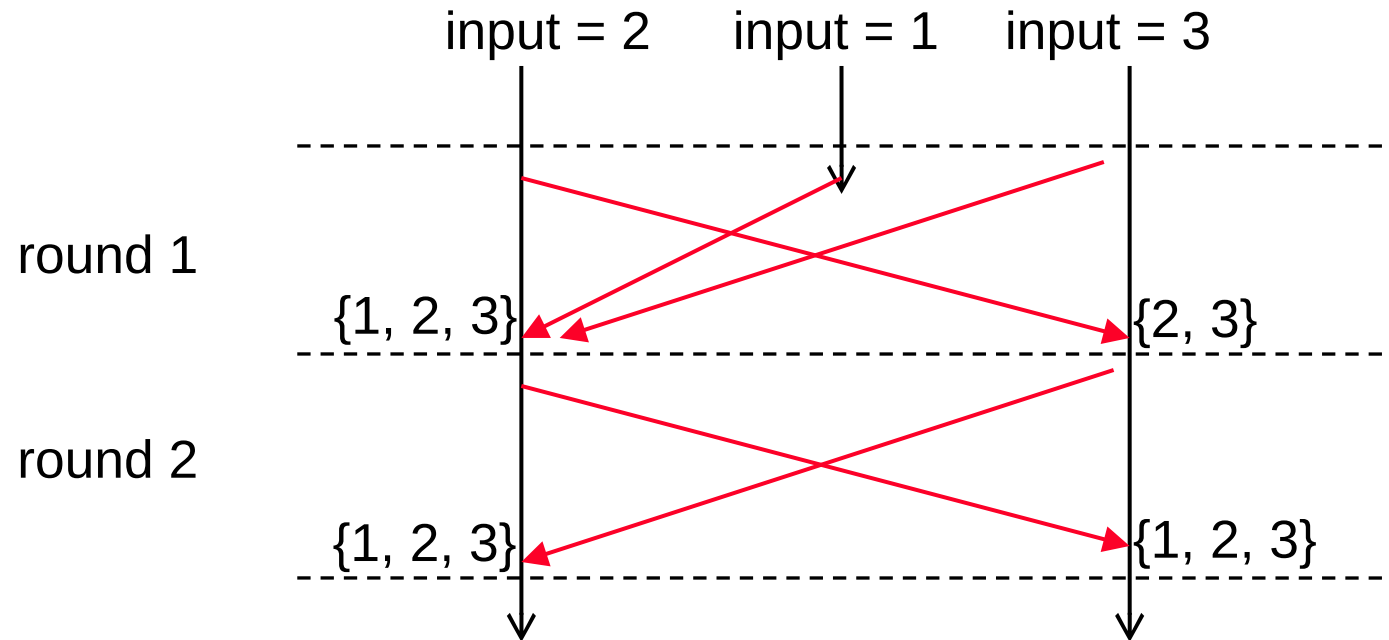
Distributed Consensus Version 1: Protocol Intuition

- Intuition: How do we do consensus if there are no failures
 - Each process sends its input to all others
 - Pick the min (or max)
- What's the problem if there are failures?
 - A process may send its input to only a subset of the processes
- Solution: Keep forwarding the values

Distributed Consensus Version 1: Protocol



Distributed Consensus Version 1: Protocol



Need one extra round for each failure ! $\Rightarrow f+1$ rounds to tolerate f failures

f needs to be an input to the protocol --- namely, the protocol needs the user to indicate the maximum number of failures to be tolerated

Distributed Consensus Version 1: Protocol

- Each node does the following:

Set S initialized to {my input}

for (int $i = 1$; $i \leq f+1$; $i++$) {

 // do this for $f+1$ rounds

 send S to all other nodes

 receive $n-1$ sets;

 for each set T received: set $S = S \cup T$

}

Decide on $\min(S)$;

Distributed Consensus Version 1: Correctness Proof for the Protocol

- **Termination**: Obvious since there is no wait
- **Validity**: Obvious since S will have only a single element
- **Agreement**: We will prove that S is the same on all nodes when the protocol terminates (the proof below is different from the proof in the textbook and is slightly easier to understand)
- A node is **nonfaulty** during round r if it has not crashed by the end of round r
- A round is **good** if there is no node failure during that round
- With $f+1$ round and f failures, there must be at least one good round

Distributed Consensus Version 1: Correctness Proof for the Protocol

- Claim: At the end of any good round r , all non-faulty nodes during round r have the same S
- Claim: Suppose r is a good round, and consider any round t after round r . During round t , the value of S on any non-faulty nodes does not change.
- Claim: All nonfaulty processes at round $f+1$ will have the same S

Distributed Consensus Version 1: Lower Bound on the Number of Rounds

- Theorem: With f crash failures, any consensus protocol will take $\Omega(f)$ rounds.
 - In fact, any consensus protocol will take at least $f+1$ rounds.
 - The proof of this theorem is beyond the scope of this module.

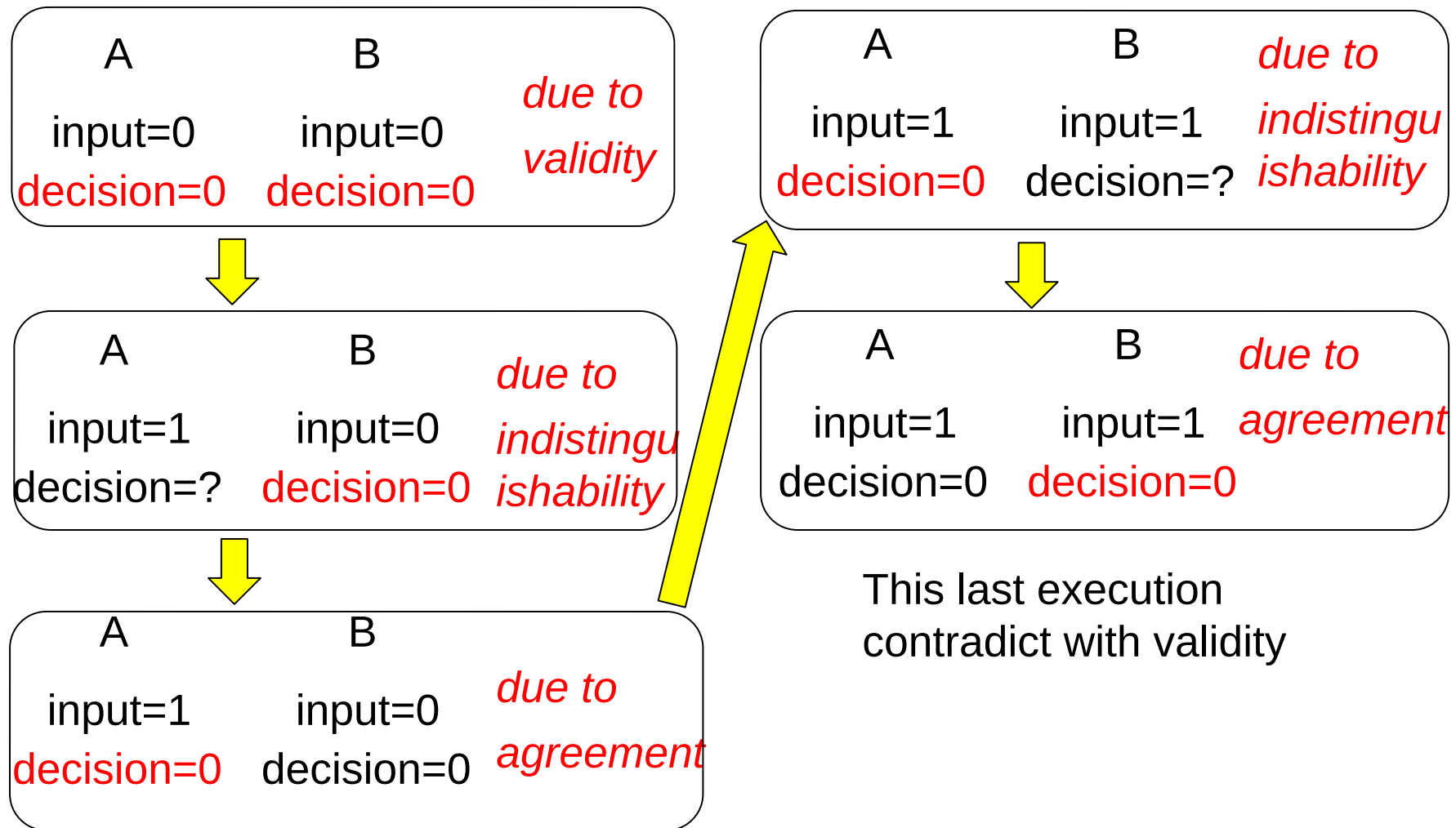
Distributed Consensus Version 2: Consensus with Link Failures

- (Also called the **coordinated attack problem**)
- Failure model:
 - Nodes do not fail
 - Communication channels may fail – may drop **arbitrary (unbounded)** number of messages
- Timing model
 - **Synchronous**: Message delay has a **known** upper bound x , and node processing delay has a **known** upper bound y

Distributed Consensus Version 2: Consensus with Link Failures

- Goal:
 - **Termination**: All nodes eventually decide
 - **Agreement**: All nodes decide on the same value
 - **Validity**: If all nodes have the same initial input, they should all decide on that. Otherwise nodes are allowed to decide on anything
- It is **impossible** to reach the above goal using a **deterministic** algorithm – since the communication channel can drop all messages
 - Trivial to prove, but the technique is important – see next slide
 - Execution α is **indistinguishable** from execution β for some node if the node sees the same things (messages and inputs) in both executions

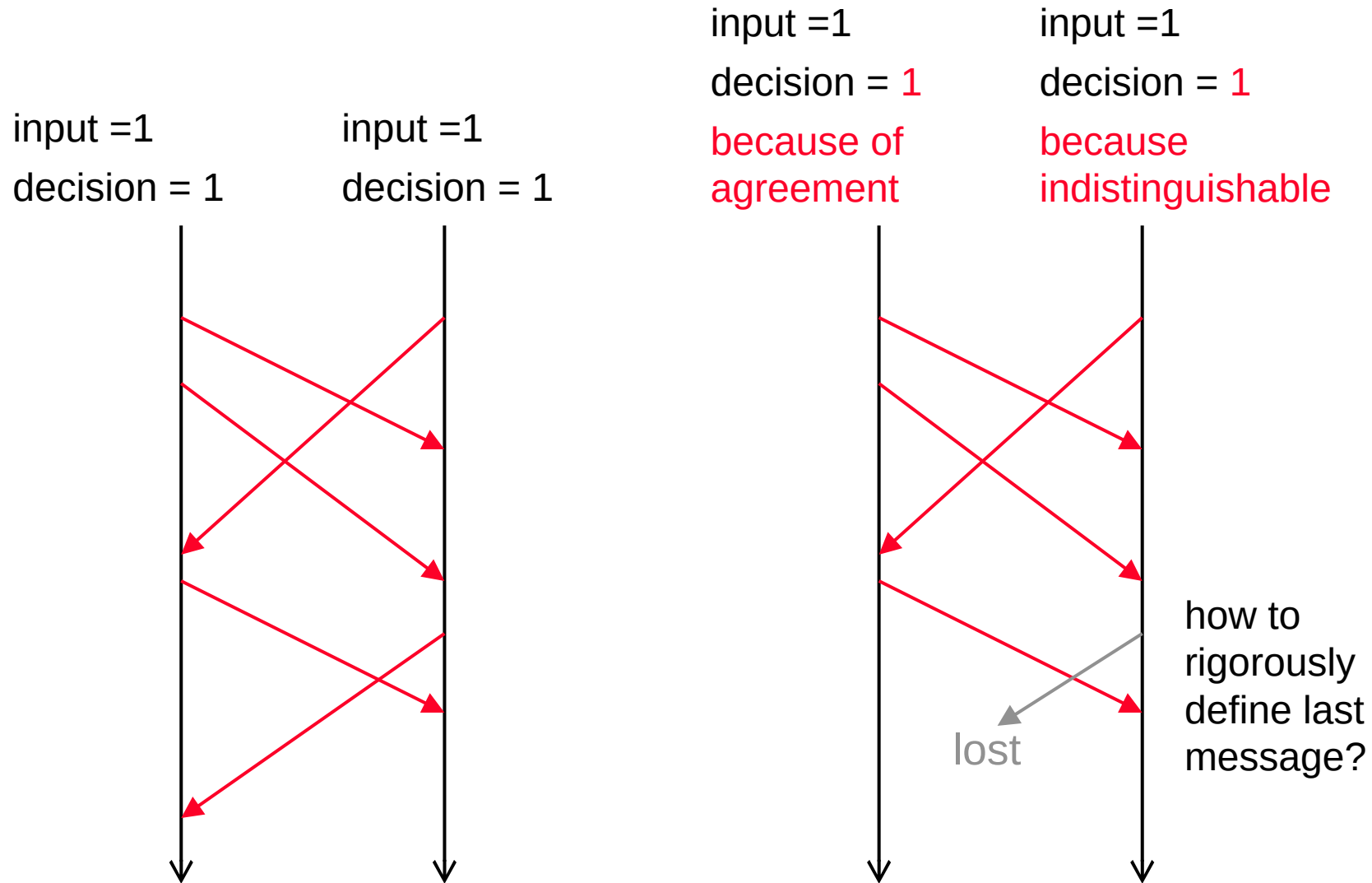
Impossibility Proof via Contradiction



Distributed Consensus Version 2: Consensus with Link Failures

- But what if we weaken our goal?
- Weakened **Validity**: Assuming input either 0 or 1
 - If all nodes start with 0, decision should be 0
 - If all nodes start with 1 **and no message is lost throughout the execution**, decision should be 1
 - Otherwise nodes are allowed to decide on anything
- Theorem: It is still **impossible** to reach the above weakened goal using a **deterministic** algorithm

Impossibility Proof via Contradiction



Distributed Consensus Version 2: Allowing Limited Disagreement

- Goal:
 - **Termination**: All nodes eventually decide
 - **Agreement**: All nodes decide on the same value with probability of $(1 - \text{error_prob})$
 - **Validity**:
 - If all nodes start with 0, decision should be 0
 - If all nodes start with 1 **and no message is lost throughout the execution**, decision should be 1
 - Otherwise nodes are allowed to decide on anything

Distributed Consensus Version 2: More Precise Notion of Randomization

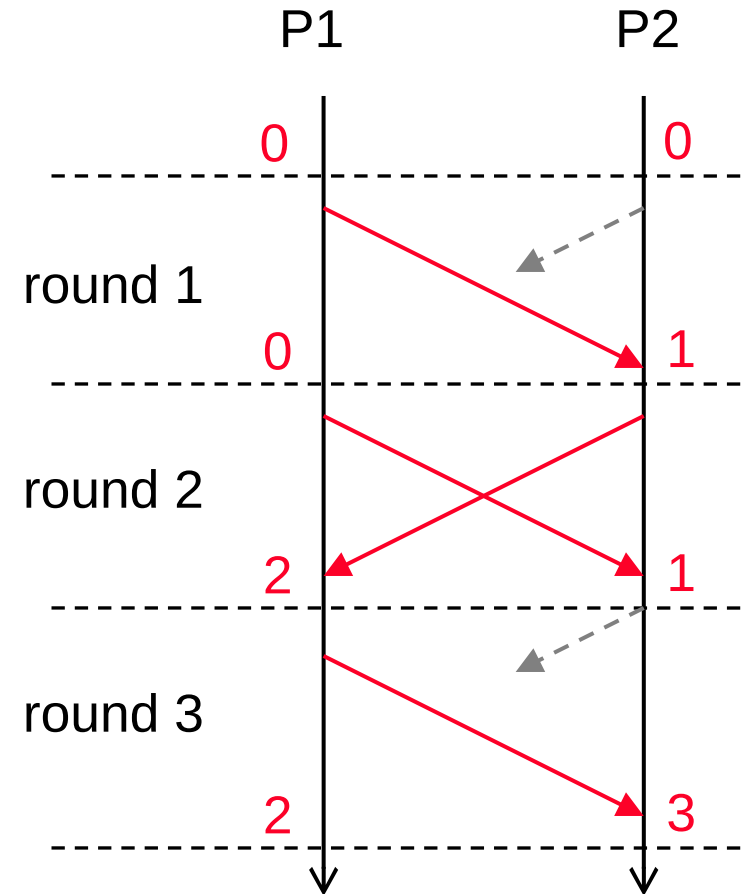
- Processes can flip coins on the fly
- **Adversary** tries to maximize error_prob by:
 - Setting the inputs of the processes
 - Cause message losses
 - In reality, there is no human adversary – we are trying to best the worst case scenario (i.e., where the adversary is not only intelligent but in fact the toughest)
- The outcomes of random coin flips are not known by adversary beforehand
 - The key to beat the adversary

A Randomized Algorithm

- For simplicity, consider two processes (can generalize to multiple): P1 and P2
- Algorithm has a predetermined number (r) of rounds
- Adversary determines which messages get lost, **before seeing the random choices**
- P1 picks a random integer **bar** $\in [1...r]$ at the beginning
- The protocol allows P1 and P2 to each maintain a **level** variable (**L1** and **L2**), such that
 - level is influenced by the adversary, but
 - L1 and L2 can differ by at most 1

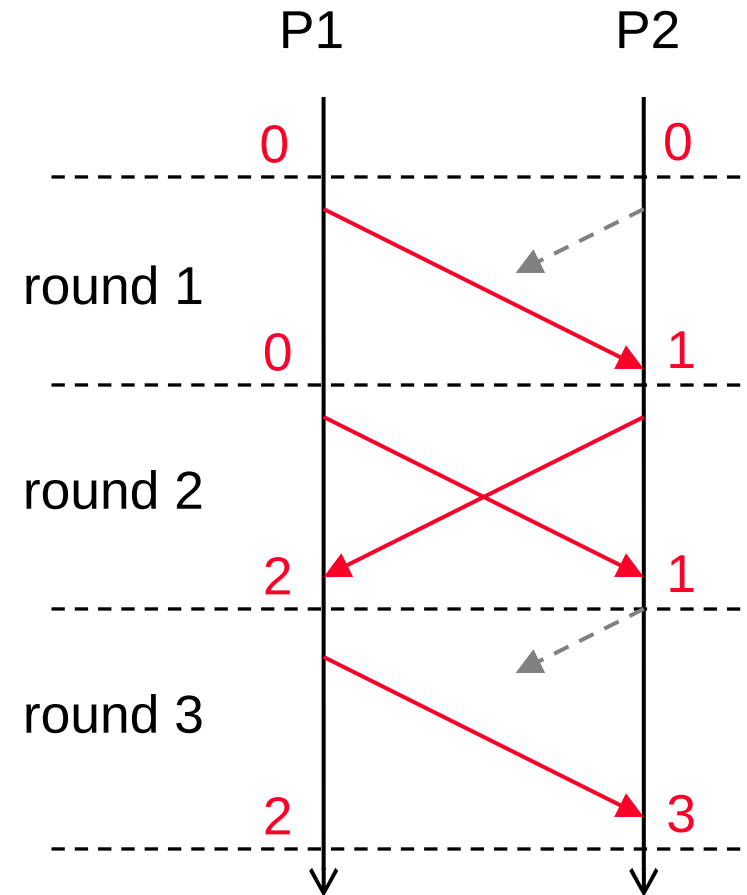
Simple Algorithm to Maintain Level

- P1 sends msg to P2 each round
- P1 sends msg to P2 each round
- bar, input and current level attached on all messages
- Upon P2 receiving a msg with L1 attached: P2 sets $L2 = L1 + 1$
- L1 maintained similarly

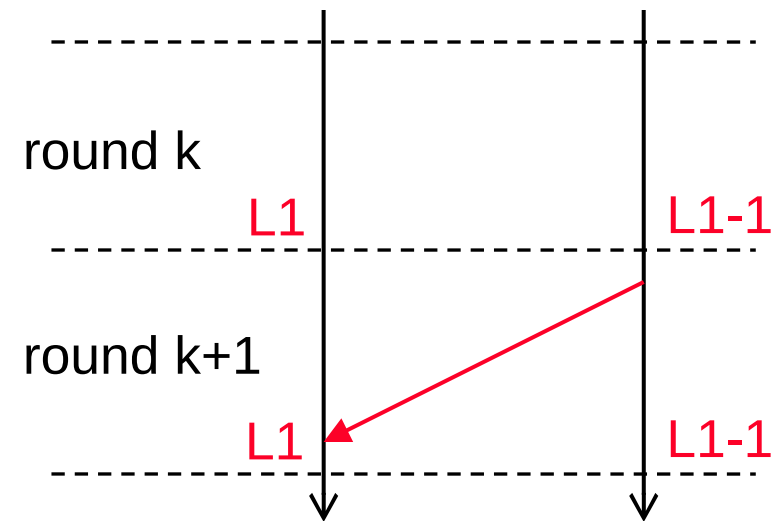
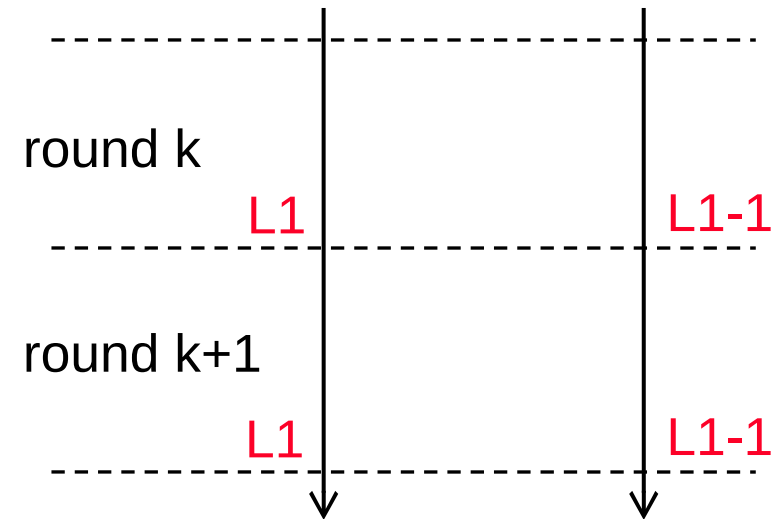
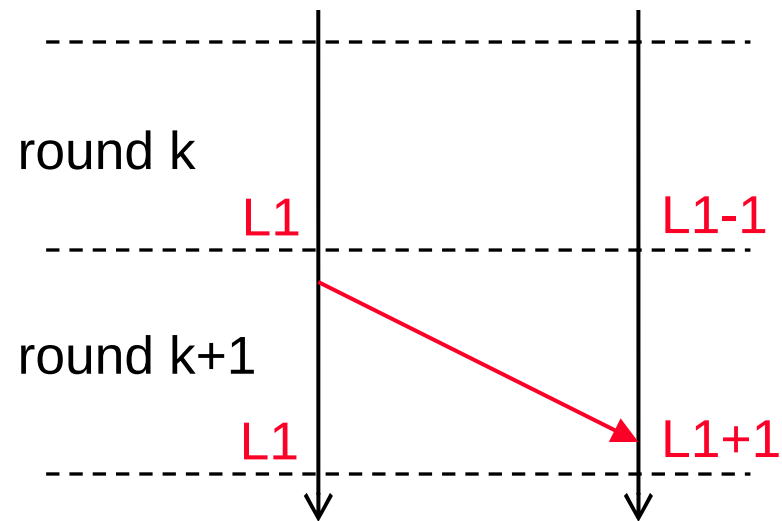
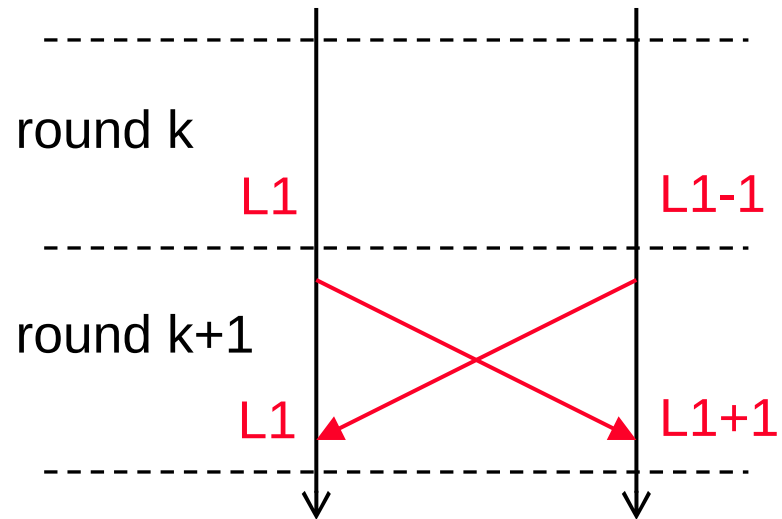


Simple Algorithm to Maintain Level

- Lemma: L1 and L2 never decreases in any round, and at the end of any round, L1 and L2 differ by at most 1.



Inductive Proof for the Lemma



Decision Rule

- At the end of the r rounds, P1 decides on 1 iff
 - P1 knows that P1's input and P2's input are both 1, and
 - $L1 \geq \text{bar}$
- (This implies that P1 will decide on 0 if it does not see P2's input.)

- At the end of the r rounds, P2 decides on 1 iff
 - P2 knows that P1's input and P2's input are both 1, and
 - P2 knows bar , and
 - $L2 \geq \text{bar}$
- (This implies that P2 will decide on 0 if it does not see P1's input or if it does not see bar .)

When does error occur?

- For P1 and P2 to decide on different values: One must decide on 1 while the other decide on 0
 - For someone to decide on 1, P1's input and P2's input must be both 1
- Case 1:
 - P1 sees P2's input, but P2 does not see P1's input or does not see bar
 - Then $L1 = 1$ and $L2 = 0$. Error occurs only when $\text{bar} = 1$.
- Case 2:
 - P2 sees P1's input and bar, but P1 does not see P2's input
 - Then $L1 = 0$ and $L2 = 1$. Error occurs only when $\text{bar} = 1$.
- Case 3:
 - P1 sees P2's input, and P2 sees P1's input and bar
 - Define $L_{\max} = \max(L1, L2)$
 - Error occurs only when $\text{bar} = L_{\max}$.

Correctness Proof

- Termination: Obvious (r rounds)
- Validity:
 - If all nodes start with 0, decision should be 0 – obvious
 - If all nodes start with 1 and no message is lost throughout the execution, decision should be 1 – If no messages are lost, then $L1=L2=r \Rightarrow P1$ and $P2$ will decide on 1
 - Otherwise nodes are allowed to decide on anything
- Agreement: With $(1-1/r)$ probability – by arguments on previous slide

Summary

<u>Failure Model and Timing Model</u>	<u>Consensus Protocol</u>
Ver 0: No node or link failures	Trivial – all-to-all broadcast
Ver 1: Node crash failures; Channels are reliable; Synchronous;	$(f+1)$ -round protocol can tolerate f crash failures
Ver 2: No node failures; Channels may drop messages (the coordinated attack problem)	Impossible without error Randomized algorithm with $1/r$ error prob
Ver 3: Node crash failures; Channels are reliable; Asynchronous;	Next lecture
Ver 4: Node Byzantine failures; Channels are reliable; Synchronous; (the Byzantine Generals problem)	Next lecture

Homework Assignment

- In the randomized algorithm for the coordinated attack problem, suppose that $r = 3$ and the adversary knows that the bar chosen by P1 is 2. Construct a scenario where the adversary ensures disagreement between the two processes by carefully selecting certain messages to drop
- Show by an example that if the consensus algorithm (for Version 1) decided the final value after f rounds instead of $f+1$ rounds, then it might violate the agreement property.
- Homework due a week from today